

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO TỔNG KẾT ĐỒ ÁN
HỌC PHẦN: PROJECT 2

Đề tài:

Hệ thống giám sát nồng độ khí Metan (CH_4) và các chỉ số môi trường hầm lò sử dụng ESP32

Sinh viên thực hiện: Lê Hoàng Ánh Nguyệt - 20236050
Giảng viên hướng dẫn: TS. Nguyễn Thị Oanh
Mã lớp học phần: IT3930

Hà Nội, ngày 1 tháng 7 năm 2026

Tóm tắt (Abstract)

Khai thác hầm lò là một trong những môi trường làm việc khắc nghiệt và tiềm ẩn nhiều rủi ro nhất. Sự rò rỉ và tích tụ của khí Metan (CH_4) là nguyên nhân hàng đầu gây ra các vụ cháy nổ hầm mỏ tàn khốc. Đồ án này tập trung nghiên cứu, thiết kế và chế tạo một hệ thống giám sát vi khí hậu hầm lò theo thời gian thực dựa trên nền tảng vi điều khiển đa nhân ESP32.

Hệ thống được thiết kế để thu thập dữ liệu liên tục từ cảm biến khí Metan (MQ-4) và cảm biến vi khí hậu (DHT22). Đặc biệt, đề tài giải quyết triệt để bài toán "nghẽn cổ chai" trong xử lý đa tác vụ của các hệ thống IoT truyền thống bằng cách áp dụng kiến trúc phần mềm không đồng bộ (Non-blocking Architecture) dựa trên bộ đếm thời gian `millis()`. Hệ thống có khả năng đưa ra cảnh báo tại chỗ (âm thanh, ánh sáng) với độ trễ xấp xỉ bằng không, đồng thời cung cấp một máy chủ Web cục bộ (Local Web Server) tích hợp bộ tổng hợp âm thanh cảnh báo thông minh đồng bộ hoàn toàn với còi cảnh báo vật lý trên phần cứng thông qua Web Audio API, cho phép người quản lý giám sát dữ liệu và điều khiển ngắt khẩn cấp hệ thống thông qua giao diện Dashboard trực quan.

Mục lục

Tóm tắt (Abstract)	1
1 Mở đầu	3
1.1 Bối cảnh và tính cấp thiết của đề tài	3
1.2 Bài toán đặt ra	3
1.3 Mục tiêu và phạm vi nghiên cứu	3
1.4 Thông số kỹ thuật chi tiết của các module	3
1.4.1 Vi điều khiển trung tâm ESP32	3
1.4.2 Cảm biến khí Gas MQ-4	4
1.4.3 Cảm biến Vi khí hậu DHT22	4
1.5 Giao thức I2C và Màn hình LCD	5
2 Thiết kế chi tiết hệ thống	5
2.1 Sơ đồ nguyên lý và Quy hoạch chân (Pinout)	5
2.2 Kiến trúc phần mềm và Thuật toán điều khiển	6
2.3 Thiết kế giao diện Web Server và API	6
3 Triển khai mã nguồn lõi và tối ưu hóa hệ thống (Implementation)	6
3.1 Cơ chế Non-blocking thay thế Delay và Phản hồi LCD tức thời	6
3.2 Ngăn ngừa phân mảnh bộ nhớ Heap bằng định dạng chuỗi tĩnh	7
3.3 Tối ưu hóa hiển thị LCD chống nhấp nháy (Flicker-Free)	8
3.4 Bộ tổng hợp âm thanh Web Audio API đồng bộ với phần cứng	9
4 Thực nghiệm và Đánh giá	10
4.1 Kịch bản thực nghiệm	10
4.2 Kết quả thực nghiệm mô phỏng trên Wokwi	11
4.2.1 Trạng thái An toàn ($MQ-4 < 500$)	12
4.2.2 Trạng thái Cảnh báo ($500 \leq MQ-4 \leq 1500$)	12
4.2.3 Trạng thái Nguy hiểm ($MQ-4 > 1500$)	12
4.3 Đánh giá ưu nhược điểm	13
5 Kết luận và Hướng phát triển	13
5.1 Kết luận	13
5.2 Hướng phát triển (Tích hợp RS-485)	13
Phụ lục: Toàn bộ mã nguồn hệ thống	15

1 Mở đầu

1.1 Bối cảnh và tính cấp thiết của đề tài

Trong quá trình khai thác than, khí Metan (CH_4) bị kẹt trong các vỉa than liên tục được giải phóng. CH_4 là một loại khí không màu, không mùi, nhẹ hơn không khí và rất dễ cháy. Khi nồng độ CH_4 trong không khí hầm lò đạt ngưỡng từ 5% đến 15%, chỉ cần một tia lửa điện nhỏ phát ra từ máy móc cũng đủ để tạo ra một vụ nổ thảm khốc. Các phương pháp đo lường thủ công truyền thống hiện nay thường mang tính chu kỳ, thiếu tính liên tục, do đó không đáp ứng được yêu cầu cảnh báo sớm. Việc áp dụng công nghệ Internet vạn vật (IoT) để tự động hóa quá trình quan trắc là một nhu cầu cấp thiết.

1.2 Bài toán đặt ra

Xây dựng một Node mạng cảm biến (Sensor Node) hoạt động độc lập, có độ tin cậy cao. Node này phải đáp ứng hai yêu cầu khắt khe:

- **Về phần cứng:** Chống chịu được môi trường nhiễu, tích hợp cơ chế cảnh báo đa phương thức (thị giác và thính giác).
- **Về phần mềm:** Xử lý đa luồng (Multitasking) hiệu quả. Hệ thống không được phép bị treo hay trễ nhịp cảnh báo khi có kết nối từ bên ngoài truy cập vào Web Server để đọc dữ liệu.

1.3 Mục tiêu và phạm vi nghiên cứu

Mục tiêu: Phát triển một bản mẫu (Prototype) hệ thống cảnh báo Metan hoàn chỉnh, có khả năng phân tích ngưỡng an toàn, luân phiên hiển thị thông số và cung cấp API giao tiếp trực tiếp với trình duyệt Web.

Phạm vi: Đồ án giới hạn trong việc thiết kế mô hình thử nghiệm, xử lý tín hiệu từ cảm biến đo môi trường và xây dựng kiến trúc phần mềm lõi trên vi điều khiển ESP32.

1.4 Thông số kỹ thuật chi tiết của các module

1.4.1 Vi điều khiển trung tâm ESP32

So với họ vi điều khiển 8-bit truyền thống như ATmega328P trên Arduino Uno, ESP32 vượt trội hoàn toàn về khả năng tính toán và giao tiếp mạng, rất phù hợp cho các node mạng IoT công nghiệp.

Bảng 1: Thông số kỹ thuật chi tiết của ESP32

Đặc tính	Thông số kỹ thuật
Vi xử lý	Tensilica Xtensa Dual-Core 32-bit LX6
Tốc độ xung nhịp	160 MHz hoặc 240 MHz
Bộ nhớ	520 KB SRAM, 4 MB Flash (Tùy phiên bản)
Kết nối Không dây	Wi-Fi: 802.11 b/g/n, Bluetooth: v4.2 BR/EDR và BLE
Giao tiếp ngoại vi	3 × UART, 3 × SPI, 2 × I2C, 16 × PWM
Điện áp hoạt động	2.2V - 3.6V (Module dev board dùng 5V qua cổng USB)

1.4.2 Cảm biến khí Gas MQ-4

Cảm biến MQ-4 có độ nhạy rất cao với khí Metan (CH_4) và Khí tự nhiên nén (CNG), trong khi ít nhạy cảm với cồn và khói, giúp giảm thiểu hiện tượng báo động giả trong môi trường hầm lò.

Bảng 2: Thông số kỹ thuật của cảm biến MQ-4

Đặc tính	Thông số kỹ thuật
Khí phát hiện mục tiêu	Metan (CH_4), Khí tự nhiên nén (CNG)
Dải nồng độ phát hiện	300 ppm đến 10,000 ppm
Điện áp làm việc (V_c)	5.0V $\pm$ 0.1V AC hoặc DC
Điện áp sưởi (V_h)	5.0V $\pm$ 0.1V AC hoặc DC
Trở kháng tải (R_L)	Có thể điều chỉnh (thường dùng 20k Ω)
Điều kiện môi trường	Nhiệt độ: -10°C đến 50°C, Độ ẩm: $\leq$ 95% RH

1.4.3 Cảm biến Vi khí hậu DHT22

Sự kết hợp giữa độ ẩm và nhiệt độ cao trong hầm lò làm tăng nguy cơ phát cháy của khí Metan. Do đó, DHT22 được sử dụng để giám sát liên tục hai chỉ số này.

Bảng 3: Đặc tính đo lường của DHT22 (AM2302)

Đặc tính	Thông số kỹ thuật
Nguồn điện	3.3V - 5.5V DC
Dải đo Nhiệt độ	-40°C đến 80°C (Độ phân giải 0.1°C, Sai số $\pm$ 0.5°C)
Dải đo Độ ẩm	0% đến 100% RH (Độ phân giải 0.1% RH, Sai số $\pm$ 2% RH)
Chu kỳ lấy mẫu	Lớn hơn hoặc bằng 2 giây (0.5 Hz)
Giao thức truyền	Tín hiệu số 1-Wire (Đường truyền đơn)

1.5 Giao thức I2C và Màn hình LCD

Để tiết kiệm chân GPIO trên ESP32, hệ thống sử dụng module chuyển đổi I2C cho màn hình LCD 16x2. Giao thức I2C (Inter-Integrated Circuit) chỉ sử dụng 2 dây tín hiệu: SDA (Serial Data) và SCL (Serial Clock), với địa chỉ vật lý mặc định thường là 0x27.

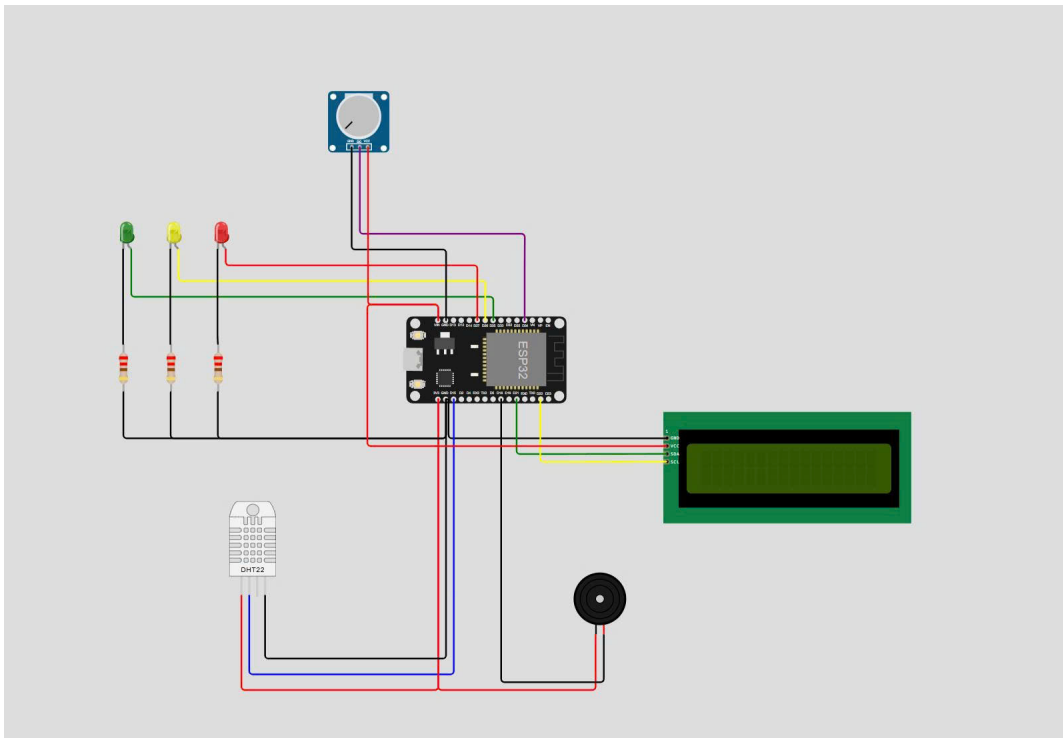
2 Thiết kế chi tiết hệ thống

2.1 Sơ đồ nguyên lý và Quy hoạch chân (Pinout)

Hệ thống được thiết kế để kết nối tối ưu với các ngoại vi. Bảng dưới đây mô tả quy hoạch chân (Pin Mapping) trên thiết bị ESP32:

Bảng 4: Quy hoạch sơ đồ kết nối ngoại vi ESP32

Tên linh kiện	Chân trên ESP32	Chức năng / Kiểu tín hiệu
Cảm biến DHT22	GPIO 15	Digital Input (1-Wire)
Cảm biến MQ-4	GPIO 34	Analog Input (ADC 12-bit)
Màn hình LCD I2C	GPIO 21 (SDA), 22 (SCL)	I2C Bus
Còi Buzzer	GPIO 18	Digital Output (Active Low)
LED Xanh (Safe)	GPIO 25	Digital Output
LED Vàng (Warning)	GPIO 26	Digital Output
LED Đỏ (Danger)	GPIO 27	Digital Output



Hình 1: Sơ đồ nguyên lý kết nối các linh kiện ngoại vi với ESP32 trên mô phỏng Wokwi

2.2 Kiến trúc phần mềm và Thuật toán điều khiển

Kiến trúc phần mềm được chia thành các phân hệ hoạt động độc lập theo thời gian (Time-sliced Multitasking).

1. **Phân hệ Cập nhật trạng thái (Logic Status):** Dữ liệu từ MQ-4 được phân ngưỡng liên tục. Nếu giá trị ADC < 500 , hệ thống ở trạng thái AN TOÀN. Từ 500 đến 1500, trạng thái CẢNH BÁO. Lớn hơn 1500, trạng thái NGUY HIỂM.
2. **Phân hệ hiển thị đa hợp (Multiplexed Display):** Do màn hình LCD 16x2 chỉ có 32 ký tự, hệ thống được thiết kế luân phiên đổi trang màn hình mỗi 2 giây: Trang 1 hiển thị Metan; Trang 2 hiển thị Nhiệt độ/Độ ẩm.

2.3 Thiết kế giao diện Web Server và API

Thay vì sử dụng các nền tảng trung gian, ESP32 tự đóng vai trò là một Access Point (WiFi SoftAP) với SSID ESP32-Metan-LeHoangAnhNguyet. Hệ thống cung cấp các RESTful API endpoints:

- GET /: Trả về mã nguồn HTML/CSS/JS.
- GET /data: Trả về cấu trúc JSON chứa số liệu real-time.
- GET /start và GET /stop: API điều khiển trạng thái hệ thống.

3 Triển khai mã nguồn lõi và tối ưu hóa hệ thống (Implementation)

Trong thiết kế phần mềm nhúng của đồ án này, các giải pháp kỹ thuật đã được tối ưu hóa sâu nhằm giải quyết triệt để các hạn chế về tài nguyên hệ thống (RAM, Heap) và trải nghiệm giao tiếp ngoại vi (LCD nhiều/nhấp nháy).

3.1 Cơ chế Non-blocking thay thế Delay và Phản hồi LCD tức thời

Hệ thống loại bỏ hoàn toàn các hàm `delay()` gây nghẽn tiến trình để tích hợp cơ chế lập lịch đa tác vụ dựa trên biến đếm `millis()`.

Ngoài ra, nhằm nâng cao tính nhạy bén của hệ thống, tác vụ cập nhật LCD (`updateLCD()`) được đồng bộ hóa để kích hoạt ngay lập tức khi bộ đọc cảm biến hoàn tất chu kỳ đọc (1 giây). Giải pháp này rút ngắn thời gian cập nhật hiển thị so với việc chỉ chờ luân phiên màn hình (2 giây):

```

1 unsigned long currentMillis = millis();
2
3 // 1. Task Doc cam bien (Chu ky 1000ms)
4 if (currentMillis - lastSensorReadTime >= sensorReadInterval) {
5     lastSensorReadTime = currentMillis;
6     mq4Value = analogRead(mq4Pin);
7
8     float newT = dht.readTemperature();
9     float newH = dht.readHumidity();
10    if (!isnan(newT)) t = newT;
11    if (!isnan(newH)) h = newH;
12
13    updateSystemStatus();
14
15    // Cap nhat LCD ngay khi co du lieu moi tu cam bien
16    if (systemOn) {
17        updateLCD();
18    }
19 }

```

3.2 Ngăn ngừa phân mảnh bộ nhớ Heap bằng định dạng chuỗi tĩnh

Ở các hệ thống IoT chạy liên tục, việc tạo và giải phóng liên tục các đối tượng thuộc lớp `String` (thông qua phép cộng chuỗi tự do) là nguyên nhân hàng đầu gây ra phân mảnh bộ nhớ Heap, dẫn đến việc thiết bị tự khởi động lại (Watchdog Reset) sau vài ngày hoạt động.

Để khắc phục hiện tượng này, hàm đóng gói JSON `getStatusJson()` đã được tối ưu hóa bằng cách sử dụng vùng đệm tĩnh cố định kích thước (`char jsonBuffer[256]`) phối hợp với hàm `snprintf()` để định dạng chuỗi an toàn:

```

1 String getStatusJson() {
2     char jsonBuffer[256];
3     const char* status_text = systemOn ? currentStatusText.c_str() :
4         "(He thong tat)";
5     const char* status_class = systemOn ? currentStatusClass.c_str()
6         : "off";
7
8     char val_str[8];

```

```

7   if (systemOn) snprintf(val_str, sizeof(val_str), "%d", mq4Value)
      ;
8   else strcpy(val_str, "---");
9
10  char temp_str[8];
11  if (systemOn && !isnan(t)) snprintf(temp_str, sizeof(temp_str),
      "%.1f", t);
12  else strcpy(temp_str, "---");
13
14  char hum_str[8];
15  if (systemOn && !isnan(h)) snprintf(hum_str, sizeof(hum_str), "
      %.1f", h);
16  else strcpy(hum_str, "---");
17
18  snprintf(jsonBuffer, sizeof(jsonBuffer),
19           "{ \"value\": \"%s\", \"text\": \"%s\", \"class\": \"%s\", \"
      temp\": \"%s\", \"hum\": \"%s\", \"on\": %s }",
20           val_str, status_text, status_class, temp_str, hum_str,
      systemOn ? "true" : "false");
21
22  return String(jsonBuffer);
23 }

```

3.3 Tối ưu hóa hiển thị LCD chống nhấp nháy (Flicker-Free)

Kỹ thuật truyền thống thường gọi lệnh xóa màn hình `lcd.clear()` hoặc in các khoảng trắng bù để làm sạch dữ liệu cũ trước khi ghi giá trị mới. Việc này tạo ra hiện tượng nhấp nháy màn hình rất mạnh và gây khó chịu khi nhìn trực tiếp.

Giải pháp tối ưu được áp dụng ở đây là xây dựng các chuỗi hiển thị có độ rộng cố định luôn đúng bằng ****16 ký tự**** trước khi gửi tới LCD. Sử dụng từ khóa định dạng căn lề trái như `%-16s`, `%-5d`, `%-4s` giúp dữ liệu mới tự động ghi đè và xóa hoàn toàn dấu vết của ký tự cũ mà không cần xóa màn hình vật lý:

```

1 void updateLCD() {
2   char line0[17];
3   char line1[17];
4
5   if (!systemOn) {
6     snprintf(line0, sizeof(line0), "%-16s", "He thong: OFF");
7     snprintf(line1, sizeof(line1), "%-16s", " (Da ngat)");

```

```

8     lcd.setCursor(0, 0); lcd.print(line0);
9     lcd.setCursor(0, 1); lcd.print(line1);
10    return;
11 }
12
13 if (showGasScreen) {
14     snprintf(line0, sizeof(line0), "Khi Metan: %-5d", mq4Value);
15     snprintf(line1, sizeof(line1), "%-16s", currentStatusLcd.c_str
        ());
16 } else {
17     char tempStr[8]; char humStr[8];
18     if (isnan(t)) strcpy(tempStr, "--"); else snprintf(tempStr,
        sizeof(tempStr), "%.1f", t);
19     if (isnan(h)) strcpy(humStr, "--"); else snprintf(humStr,
        sizeof(humStr), "%.1f", h);
20
21     snprintf(line0, sizeof(line0), "Nhiệt độ: %-4sC", tempStr);
22     snprintf(line1, sizeof(line1), "Độ ẩm:      %-4s%%", humStr);
23 }
24
25 lcd.setCursor(0, 0); lcd.print(line0);
26 lcd.setCursor(0, 1); lcd.print(line1);
27 }

```

3.4 Bộ tổng hợp âm thanh Web Audio API đồng bộ với phần cứng

Để tối ưu hóa trải nghiệm giám sát từ xa, giao diện Dashboard Web được trang bị hệ thống âm thanh cảnh báo thời gian thực sử dụng thư viện xử lý âm thanh số **Web Audio API** tích hợp sẵn của trình duyệt.

Để đảm bảo tín hiệu cảnh báo trên Web khớp hoàn toàn với thiết bị vật lý tại hầm lò, một sơ đồ máy trạng thái âm thanh đã được xây dựng tại Frontend. Bộ quản lý này sẽ lọc trùng lặp tín hiệu và điều khiển tần số cũng như chu kỳ dao động phù hợp:

- **Trạng thái An toàn (Safe):** Âm thanh tắt hoàn toàn (Gain = 0).
- **Trạng thái Cảnh báo (Warning):** Hệ thống khởi động một tiến trình lặp `setInterval()` với chu kỳ `**200 ms**`, nhấp nháy âm lượng tắt/bật liên tục khớp chính xác với chu kỳ ngắt quãng còi vật lý (`buzzerBeepInterval = 200`), sử dụng tần số cảnh báo trung bình 600 Hz.

- **Trạng thái Nguy hiểm (Danger):** Thiết lập còi hú liên tục không ngừng ở tần số cao 1000 Hz chói tai, khớp với cơ chế còi hú liên tục trên bo mạch phần cứng.

Giải pháp này giải quyết được vấn đề trễ tín hiệu và đảm bảo tính đồng bộ thông tin đa kênh trong toàn bộ cấu trúc mạng IoT của mô hình giám sát.

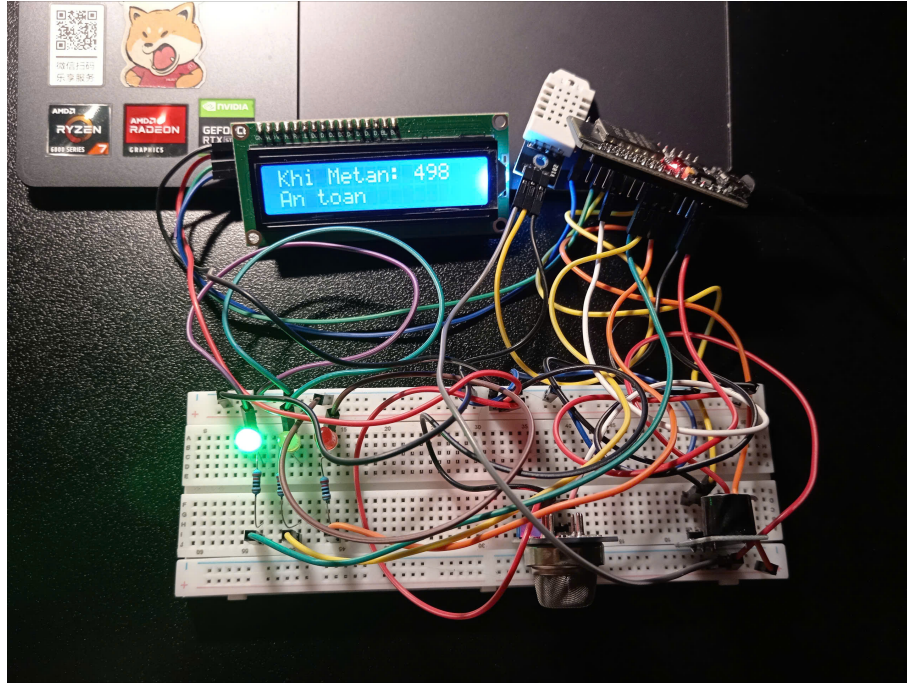
4 Thực nghiệm và Đánh giá

4.1 Kịch bản thực nghiệm

Hệ thống được cấp nguồn và đưa vào môi trường mô phỏng. Bảng dưới đây tóm tắt các kịch bản kiểm thử:

Bảng 5: Bảng đánh giá kịch bản thực nghiệm

Kịch bản	Tác động vật lý	Phản hồi của thiết bị	Web Server
1. An Toàn	Khí bình thường, không có tác nhân.	LED Xanh sáng, còi tắt. Màn hình luân phiên.	Hiển thị xanh, báo "An toàn". Âm thanh tắt.
2. Cảnh Báo	Đưa bật lửa (không xẹt lửa) ra xa cảm biến.	Giá trị ADC ≈ 800 . LED Vàng sáng. Còi bip nhịp 200ms.	Hiển thị vàng. Web phát tiếng bip ngắt quãng nhịp 200ms.
3. Nguy Hiểm	Phụt trực tiếp gas từ bật lửa vào MQ-4.	Giá trị ADC > 1500 . LED Đỏ sáng. Còi hú liên tục.	Báo đỏ. Còi Web phát âm cảnh báo kéo dài liên tục.
4. Tắt hệ thống	Nhấn nút "Ngắt" trên trình duyệt Web.	Toàn bộ LED tắt. Còi tắt. Màn hình báo "He thong: OFF".	Các thông số mờ đi. Âm thanh ngắt hoàn toàn.



Hình 2: Mô hình phần cứng thực tế sau khi đấu nối hoàn chỉnh

Để đánh giá khả năng hoạt động và độ nhạy bén của mô hình phần cứng thực tế, em đã tiến hành thử nghiệm thủ công bằng các tác nhân vật lý trực tiếp:

- **Thử nghiệm cảm biến khí gas MQ-4:** Sử dụng bật lửa gas phóng thích trực tiếp một lượng nhỏ khí butane/metan ở khoảng cách gần cảm biến để giả lập sự cố rò rỉ khí gas trong hầm lò.
- **Thử nghiệm cảm biến vi khí hậu DHT22:** Tiến hành hà hơi thở nóng ẩm trực tiếp vào bề mặt cảm biến để làm thay đổi nhanh nhiệt độ và độ ẩm, giả lập điều kiện môi trường vi khí hậu hầm lò thay đổi đột ngột.

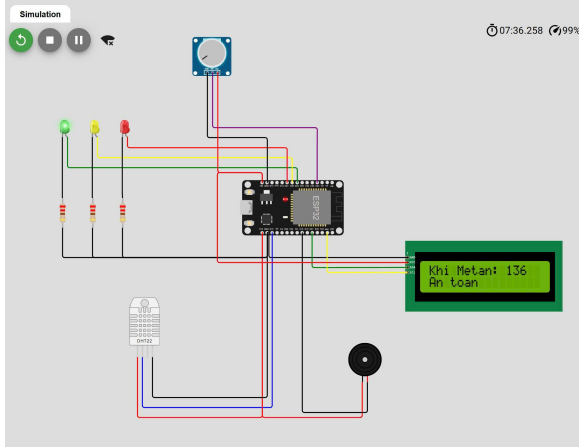
Kết quả thực nghiệm trên phần cứng thật cho thấy: Hệ thống phản hồi nhạy bén với độ trễ xấp xỉ bằng không. Khi xì gas bật lửa, giá trị đo tăng vọt vượt ngưỡng 1500, còi báo động tại chỗ kêu liên tục và đèn LED Đỏ sáng ngay lập tức. Đồng thời, khi hà hơi ẩm vào DHT22, màn hình LCD và Dashboard trên trình duyệt Web cập nhật chỉ số nhiệt độ, độ ẩm tăng dần tương ứng theo thời gian thực (chu kỳ cập nhật 1 giây).

4.2 Kết quả thực nghiệm mô phỏng trên Wokwi

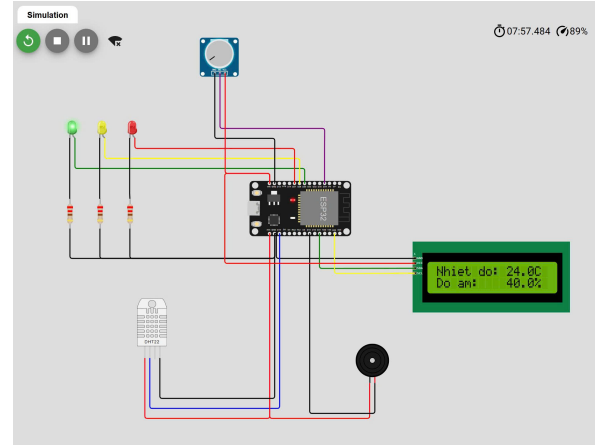
Dưới đây là kết quả thực nghiệm kiểm thử hệ thống trên môi trường mô phỏng Wokwi đối với ba trạng thái cảnh báo của hệ thống. Ở mỗi trạng thái, màn hình LCD 16x2 hiển thị luân phiên hai giao diện thông tin (Nồng độ khí Metan và vi khí hậu Nhiệt độ - Độ ẩm).

4.2.1 Trạng thái An toàn ($MQ-4 < 500$)

Ở trạng thái bình thường, nồng độ khí Metan đo được ở mức thấp. Đèn LED Xanh sáng, còi báo động tắt và hệ thống hiển thị thông tin như Hình 4.2.1.



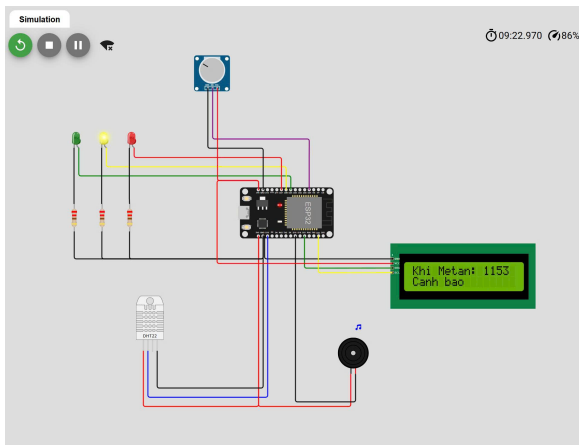
Hình 3: Màn hình hiển thị Metan (An toàn)



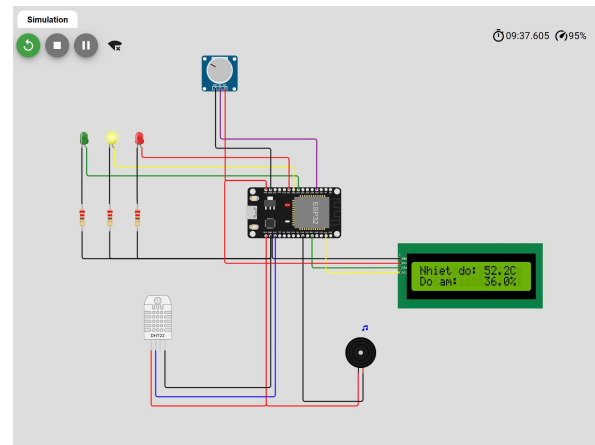
Hình 4: Màn hình hiển thị Nhiệt độ - Độ ẩm (An toàn)

4.2.2 Trạng thái Cảnh báo ($500 \leq MQ-4 \leq 1500$)

Khi nồng độ khí Metan tăng vượt ngưỡng bình thường, đèn LED Vàng lập tức sáng lên, còi phát tiếng bíp gián đoạn chu kỳ 200ms. Kết quả hiển thị như Hình 4.2.2.



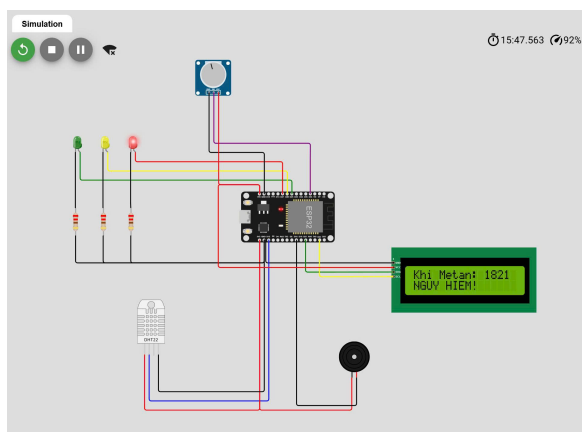
Hình 5: Màn hình hiển thị Metan (Cảnh báo)



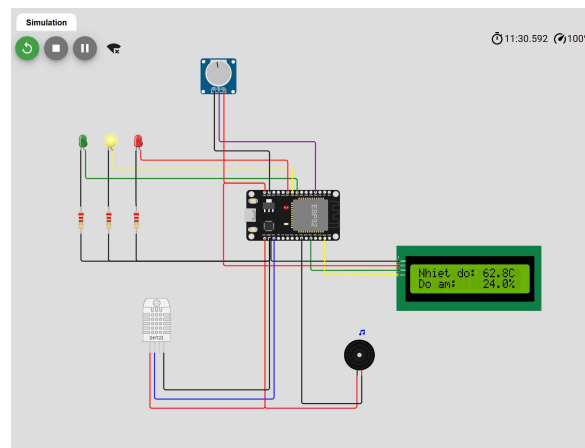
Hình 6: Màn hình hiển thị Nhiệt độ - Độ ẩm (Cảnh báo)

4.2.3 Trạng thái Nguy hiểm ($MQ-4 > 1500$)

Khi nồng độ khí Metan vượt ngưỡng nguy hiểm (có nguy cơ cháy nổ cực cao), đèn LED Đỏ sáng và còi hú liên tục không ngừng để cảnh báo công nhân sơ tán khẩn cấp. Kết quả hiển thị như Hình 4.2.3.



Hình 7: Màn hình hiển thị Metan (Nguy hiểm)



Hình 8: Màn hình hiển thị Nhiệt độ - Độ ẩm (Nguy hiểm)

4.3 Đánh giá ưu nhược điểm

Ưu điểm: Tính năng hoạt động cực kỳ mượt mà. Lỗi vi điều khiển không gặp hiện tượng quá tải (Watchdog Timer Reset). Giao diện thiết kế theo triết lý "Mobile-first" rất thân thiện. Nhờ cải tiến hiển thị tĩnh, LCD hoàn toàn không có hiện tượng giật, nháy khi đo lường. Hệ thống cảnh báo âm thanh đa kênh (tại thiết bị và trên Web Dashboard) hoạt động đồng bộ với độ trễ dưới 1 giây.

Hạn chế: Việc ánh xạ từ giá trị ADC thô (0-4095) sang đơn vị ppm chính xác tuyệt đối đòi hỏi phải đo đạc trong buồng thử nghiệm tiêu chuẩn để xác định hệ số R_0 , đây là một hạn chế về mặt thiết bị đo lường của đồ án.

5 Kết luận và Hướng phát triển

5.1 Kết luận

Đồ án đã thiết kế và chế tạo thành công một trạm quan trắc vi khí hậu mini. Điểm sáng của đề tài nằm ở việc tối ưu hóa mã nguồn nhúng, làm chủ kỹ thuật điều phối thời gian thực và xây dựng máy chủ Web Server tích hợp âm thanh cảnh báo thông minh trên một vi mạch nhỏ gọn. Hệ thống đáp ứng trọn vẹn yêu cầu của Project 2 và cung cấp một mô hình trực quan cho bài toán an toàn hầm lò.

5.2 Hướng phát triển (Tích hợp RS-485)

Như đã phân tích ở phần lý thuyết, sóng WiFi bị hạn chế khoảng cách trong hầm mỏ. Bước phát triển tiếp theo của dự án là cấu hình chân TX/RX của ESP32 kết nối với IC MAX485. Dữ liệu thay vì truyền qua WiFi sẽ được đóng gói truyền qua giao thức

Modbus RTU trên đường truyền vật lý RS-485, giúp hệ thống có thể kéo dây truyền số liệu về trạm trung tâm cách xa hàng kilomet mà không bị suy hao tín hiệu.

Phụ lục: Toàn bộ mã nguồn hệ thống (Source Code)

Đoạn mã dưới đây là toàn bộ logic điều khiển đã tối ưu dung lượng, tốc độ xử lý và tích hợp bộ tổng hợp âm thanh Web Audio API đồng bộ phần cứng được nạp vào vi điều khiển ESP32.

```
1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3 #include <WiFi.h>
4 #include <WebServer.h>
5 #include <DHT.h>
6
7 // --- CAI DAT CAM BIEN DHT ---
8 #define DHTPIN 15
9 #define DHTTYPE DHT22
10 DHT dht(DHTPIN, DHTTYPE);
11
12 // --- CAI DAT LCD VA CAC CHAN KHAC ---
13 LiquidCrystal_I2C lcd(0x27, 16, 2);
14
15 const int mq4Pin = 34;
16 const int buzzerPin = 18; // Module Active Low (HIGH=Tat, LOW=Bat)
17
18 const int ledGreen = 25;
19 const int ledYellow = 26;
20 const int ledRed = 27;
21
22 // --- CAI DAT WIFI ---
23 const char* ap_ssid = "ESP32-Metan-LeHoangAnhNguyet";
24 const char* ap_password = "12345678";
25
26 WebServer server(80);
27 bool systemOn = true;
28
29 // --- CAC BIEN TRANG THAI ---
30 int mq4Value = 0;
31 float t = 0.0;
32 float h = 0.0;
33
34 String currentStatusText = "";
35 String currentStatusLcd = "";
36 String currentStatusClass = "";
37 bool lastBuzzerState = HIGH;
38
39 // Thoi gian khong dong bo (non-blocking)
40 unsigned long lastSensorReadTime = 0;
41 const long sensorReadInterval = 1000;
42
43 unsigned long lastBuzzerBeep = 0;
44 const long buzzerBeepInterval = 200;
45
46 unsigned long lastLcdToggle = 0;
```

```

47 const long lcdToggleInterval = 2000;
48 bool showGasScreen = true;
49
50 const int mq4MaxGaugeValue = 2500;
51
52 // === HAM CAP NHAT TRANG THAI ===
53 void updateSystemStatus() {
54     if (mq4Value < 500) {
55         currentStatusText = "An toan";
56         currentStatusLcd = "An toan";
57         currentStatusClass = "safe";
58     } else if (mq4Value >= 500 && mq4Value <= 1500) {
59         currentStatusText = "Canh bao";
60         currentStatusLcd = "Canh bao";
61         currentStatusClass = "warning";
62     } else {
63         currentStatusText = "NGUY HIEM!";
64         currentStatusLcd = "NGUY HIEM!";
65         currentStatusClass = "danger";
66     }
67 }
68
69 // === HAM HIEN THI LCD LUAN PHIEN (TOI UU HOA TRANH NHAP NHAY)
70 // ===
71 void updateLCD() {
72     char line0[17];
73     char line1[17];
74
75     if (!systemOn) {
76         snprintf(line0, sizeof(line0), "%-16s", "He thong: OFF");
77         snprintf(line1, sizeof(line1), "%-16s", " (Da ngat)");
78
79         lcd.setCursor(0, 0);
80         lcd.print(line0);
81         lcd.setCursor(0, 1);
82         lcd.print(line1);
83         return;
84     }
85
86     if (showGasScreen) {
87         snprintf(line0, sizeof(line0), "Khi Metan: %-5d", mq4Value);
88         snprintf(line1, sizeof(line1), "%-16s", currentStatusLcd.c_str());
89     } else {
90         char tempStr[8];
91         char humStr[8];
92
93         if (isnan(t)) strcpy(tempStr, "--");
94         else snprintf(tempStr, sizeof(tempStr), "%.1f", t);
95
96         if (isnan(h)) strcpy(humStr, "--");

```

```

96     else snprintf(humStr, sizeof(humStr), "%.1f", h);
97
98     snprintf(line0, sizeof(line0), "Nhiet do: %-4sC", tempStr);
99     snprintf(line1, sizeof(line1), "Do am:    %-4s%%", humStr);
100 }
101
102 lcd.setCursor(0, 0);
103 lcd.print(line0);
104 lcd.setCursor(0, 1);
105 lcd.print(line1);
106 }
107
108 // == HAM TRA DU LIEU JSON CHO WEB ==
109 String getStatusJson() {
110     char jsonBuffer[256];
111
112     const char* status_text = systemOn ? currentStatusText.c_str() :
113         "(He thong tat)";
114     const char* status_class = systemOn ? currentStatusClass.c_str()
115         : "off";
116
117     char val_str[8];
118     if (systemOn) snprintf(val_str, sizeof(val_str), "%d", mq4Value)
119         ;
120     else strcpy(val_str, "---");
121
122     char temp_str[8];
123     if (systemOn && !isnan(t)) snprintf(temp_str, sizeof(temp_str),
124         "%.1f", t);
125     else strcpy(temp_str, "---");
126
127     char hum_str[8];
128     if (systemOn && !isnan(h)) snprintf(hum_str, sizeof(hum_str), "
129         %.1f", h);
130     else strcpy(hum_str, "---");
131
132     snprintf(jsonBuffer, sizeof(jsonBuffer),
133         "{\"value\\\":\\\"%s\\\", \"text\\\":\\\"%s\\\", \"class\\\":\\\"%s\\\", \"
134         temp\\\":\\\"%s\\\", \"hum\\\":\\\"%s\\\", \"on\\\":%s}\",
135         val_str, status_text, status_class, temp_str, hum_str,
136         systemOn ? "true" : "false");
137
138     return String(jsonBuffer);
139 }
140
141 // == GIAO DIEN WEB (HTML/CSS/JS + PROGMEM + WEB AUDIO API) ==
142 const char webPageHtml[] PROGMEM = R"rawliteral(
143 <!DOCTYPE html>
144 <html>
145 <head>
146 <meta charset="UTF-8">

```

```

140 <meta name="viewport" content="width=device-width, initial-scale
    =1.0">
141 <title>He thong Giam sat PRO</title>
142 <style>
143   :root {
144     --bg-color: #f4f7f6; --card-color: #ffffff;
145     --text-main: #333; --text-sub: #777;
146     --safe: #2ecc71; --warning: #f1c40f; --danger: #e74c3c;
147     --off: #bdc3c7; --info: #3498db;
148     --btn-start: #3498db; --btn-stop: #e74c3c;
149     --btn-mute: #9b59b6;
150   }
151   body {
152     font-family: 'Segoe UI', sans-serif; background-color: var(--
      bg-color);
153     margin: 0; display: flex; justify-content: center; align-items
      : center; min-height: 100vh;
154   }
155   .main-card {
156     background-color: var(--card-color); border-radius: 15px;
157     box-shadow: 0 10px 30px rgba(0,0,0,0.1); padding: 30px;
158     width: 90%; max-width: 500px; text-align: center;
159   }
160   h1 { margin-top: 0; font-size: 1.5rem; color: var(--text-main);
    }
161   p.sub-title { color: var(--text-sub); margin-top: -10px; font-
    size: 0.9rem; }
162   .gauge-container { margin: 20px 0; position: relative; }
163   .gauge {
164     position: relative; width: 150px; height: 150px; border-radius
      : 50%;
165     background: conic-gradient(var(--off) 0%, var(--bg-color) 0%);
166     margin: 0 auto; transition: background 0.3s ease;
167   }
168   .gauge-cover {
169     position: absolute; top: 12px; left: 12px; right: 12px; bottom
      : 12px;
170     background-color: var(--card-color); border-radius: 50%;
171     display: flex; justify-content: center; align-items: center;
      flex-direction: column;
172     box-shadow: inset 0px 2px 5px rgba(0,0,0,0.05);
173   }
174   .gauge-cover .val { font-size: 2rem; font-weight: bold; color:
    var(--text-main); line-height: 1;}
175   .gauge-cover .lbl { font-size: 0.8rem; color: var(--text-sub);
    margin-top: 5px;}
176   .env-grid {
177     display: grid; grid-template-columns: 1fr 1fr; gap: 15px;
      margin-bottom: 20px;
178   }
179   .env-box {

```

```

180     background: rgba(52, 152, 219, 0.1); border: 1px solid rgba
181         (52, 152, 219, 0.3);
182     border-radius: 8px; padding: 15px 10px; transition: all 0.3s
183         ease;
184 }
185 .env-box .lbl { font-size: 0.9rem; color: var(--text-sub);
186     display: block; margin-bottom: 5px; text-transform: uppercase
187     ; font-weight: 600;}
188 .env-box .val { font-size: 1.5rem; font-weight: bold; color: var
189     (--info); }
190 .env-box.off { background: rgba(189,195,199,0.1); border-color:
191     rgba(189,195,199,0.3); }
192 .env-box.off .val { color: var(--off); }
193 .status-box {
194     border-radius: 8px; padding: 15px; margin: 15px 0; font-size:
195     1.1rem;
196     font-weight: 600; text-transform: uppercase; transition: all
197     0.3s ease;
198 }
199 .safe { background-color: rgba(46, 204, 113, 0.15); color: var
200     (--safe); border: 1px solid var(--safe); }
201 .warning { background-color: rgba(241, 196, 15, 0.15); color:
202     var(--warning); border: 1px solid var(--warning); }
203 .danger { background-color: rgba(231, 76, 60, 0.15); color: var
204     (--danger); border: 1px solid var(--danger); }
205 .off { background-color: rgba(189, 195, 199, 0.15); color: var
206     (--off); border: 1px solid var(--off); }
207 .control-area { display: flex; justify-content: center; gap: 10
208     px; margin-top: 20px; flex-wrap: wrap; }
209 .btn {
210     border: none; border-radius: 5px; padding: 12px 20px; font-
211     size: 0.95rem;
212     font-weight: 600; cursor: pointer; transition: all 0.2s ease;
213     display: flex; align-items: center; gap: 8px;
214 }
215 .btn-start { background-color: var(--btn-start); color: #fff; }
216 .btn-stop { background-color: var(--btn-stop); color: #fff; }
217 .btn-mute { background-color: var(--btn-mute); color: #fff; }
218 .btn-mute.muted { background-color: var(--off); }
219 .btn:disabled { opacity: 0.5; cursor: not-allowed; }
220 </style>
221 </head>
222 <body>
223     <div class="main-card">
224         <h1>Giam sat Moi truong</h1>
225         <p class="sub-title">Khi Metan - Nhiet do - Do am</p>
226         <div class="gauge-container">
227             <div id="gauge" class="gauge">
228                 <div class="gauge-cover">
229                     <span id="value" class="val">---</span>
230                     <span class="lbl">Khi Metan</span>

```

```

216     </div>
217 </div>
218 </div>
219 <div class="env-grid">
220   <div id="temp-box" class="env-box off">
221     <span class="lbl">Nhiet do</span>
222     <span id="temp" class="val">--</span><span style="color:
      var(--text-sub); font-size:0.9rem"> &deg;C</span>
223   </div>
224   <div id="hum-box" class="env-box off">
225     <span class="lbl">Do am</span>
226     <span id="hum" class="val">--</span><span style="color:var
      (--text-sub); font-size:0.9rem"> %</span>
227   </div>
228 </div>
229 <div id="status-box" class="status-box off">
230   <span id="status-text">...</span>
231 </div>
232 <div class="control-area">
233   <button id="btn-start" class="btn btn-start" onclick="
      controlSystem('/start')" disabled>Bat</button>
234   <button id="btn-stop" class="btn btn-stop" onclick="
      controlSystem('/stop')">Ngat</button>
235   <button id="btn-mute" class="btn btn-mute" onclick="
      toggleMute()">Coi Web</button>
236 </div>
237 </div>
238 <script>
239   const gauge = document.getElementById('gauge');
240   const valueSpan = document.getElementById('value');
241   const tempSpan = document.getElementById('temp');
242   const humSpan = document.getElementById('hum');
243   const tempBox = document.getElementById('temp-box');
244   const humBox = document.getElementById('hum-box');
245   const statusBox = document.getElementById('status-box');
246   const statusTextSpan = document.getElementById('status-text');
247   const btnStart = document.getElementById('btn-start');
248   const btnStop = document.getElementById('btn-stop');
249   const btnMute = document.getElementById('btn-mute');
250   const maxGaugeValue = 2500;
251
252   let audioCtx = null;
253   let osc = null;
254   let gainNode = null;
255   let warningTimer = null;
256   let isMuted = false;
257   let currentAlarmState = "none";
258
259   function initAudio() {
260     if (!audioCtx) {
261       audioCtx = new (window.AudioContext || window.

```

```

262         webkitAudioContext)();
263         osc = audioCtx.createOscillator();
264         gainNode = audioCtx.createGain();
265
266         osc.type = 'sine';
267         osc.frequency.setValueAtTime(600, audioCtx.currentTime);
268         gainNode.gain.setValueAtTime(0, audioCtx.currentTime);
269
270         osc.connect(gainNode);
271         gainNode.connect(audioCtx.destination);
272         osc.start();
273     }
274 }
275
276 function toggleMute() {
277     initAudio();
278     if (audioCtx && audioCtx.state === 'suspended') {
279         audioCtx.resume();
280     }
281     isMuted = !isMuted;
282     if (isMuted) {
283         btnMute.innerText = "          Tat coi";
284         btnMute.classList.add('muted');
285         muteAudio();
286     } else {
287         btnMute.innerText = "          Coi Web";
288         btnMute.classList.remove('muted');
289         unmuteAudio();
290     }
291 }
292
293 function muteAudio() {
294     if (gainNode && audioCtx) {
295         gainNode.gain.setValueAtTime(0, audioCtx.currentTime);
296     }
297     if (warningTimer) {
298         clearInterval(warningTimer);
299         warningTimer = null;
300     }
301 }
302
303 function unmuteAudio() {
304     applyAlarmState(currentAlarmState);
305 }
306
307 function applyAlarmState(state) {
308     currentAlarmState = state;
309     if (isMuted || !audioCtx) return;
310
311     if (audioCtx.state === 'suspended') {
312         audioCtx.resume();
313     }

```

```

312     }
313
314     if (warningTimer) {
315         clearInterval(warningTimer);
316         warningTimer = null;
317     }
318
319     if (state === "none") {
320         gainNode.gain.setValueAtTime(0, audioCtx.currentTime);
321     }
322     else if (state === "warning") {
323         osc.frequency.setValueAtTime(600, audioCtx.currentTime);
324         let beepOn = false;
325
326         warningTimer = setInterval(() => {
327             if (isMuted) return;
328             beepOn = !beepOn;
329             gainNode.gain.setTargetAtTime(beepOn ? 0.2 : 0, audioCtx
330                 .currentTime, 0.01);
331         }, 200);
332     }
333     else if (state === "danger") {
334         osc.frequency.setValueAtTime(1000, audioCtx.currentTime);
335         gainNode.gain.setTargetAtTime(0.2, audioCtx.currentTime,
336             0.01);
337     }
338 }
339
340 function handleAlarmSound(statusClass, systemOn) {
341     if (!systemOn || statusClass === 'safe' || statusClass === '
342         off') {
343         if (currentAlarmState !== "none") {
344             applyAlarmState("none");
345         }
346     }
347     else if (statusClass === 'warning') {
348         if (currentAlarmState !== "warning") {
349             applyAlarmState("warning");
350         }
351     }
352     else if (statusClass === 'danger') {
353         if (currentAlarmState !== "danger") {
354             applyAlarmState("danger");
355         }
356     }
357 }
358
359 document.addEventListener('click', () => {
360     initAudio();
361     if (audioCtx && audioCtx.state === 'suspended') {
362         audioCtx.resume();
363     }
364 }, { once: true });

```

```

360
361 function updatePage() {
362     fetch('/data')
363     .then(res => res.json())
364     .then(data => {
365         valueSpan.innerText = data.value;
366         let percent = 0;
367         if (data.on && !isNaN(parseInt(data.value))) {
368             percent = Math.min((parseInt(data.value) /
369                 maxGaugeValue) * 100, 100);
370         }
371         let color = 'var(--off)';
372         if (data.on) {
373             if (data.class === 'safe') color = 'var(--safe)';
374             else if (data.class === 'warning') color = 'var(--
375                 warning)';
376             else if (data.class === 'danger') color = 'var(--
377                 danger)';
378         }
379         gauge.style.background = 'conic-gradient(${color} ${
380             percent}%, var(--bg-color) ${percent}%)';
381         tempSpan.innerText = data.temp;
382         humSpan.innerText = data.hum;
383         if(data.on) {
384             tempBox.classList.remove('off'); humBox.classList.
385                 remove('off');
386         } else {
387             tempBox.classList.add('off'); humBox.classList.add('
388                 off');
389         }
390         statusTextSpan.innerText = data.text;
391         statusBox.className = 'status-box ${data.class}';
392
393         handleAlarmSound(data.class, data.on);
394
395         btnStart.disabled = data.on;
396         btnStop.disabled = !data.on;
397     })
398     .catch(err => {
399         statusTextSpan.innerText = "LOI KET NOI";
400         statusBox.className = "status-box off";
401         gauge.style.background = 'conic-gradient(var(--off) 0%,
402             var(--bg-color) 0%)';
403         valueSpan.innerText = '---'; tempSpan.innerText = '--';
404         humSpan.innerText = '--';
405         tempBox.classList.add('off'); humBox.classList.add('off
406             ');
407         applyAlarmState("none");
408     });
409 }
410
411

```

```

402     function controlSystem(endpoint) {
403         btnStart.disabled = true; btnStop.disabled = true;
404         fetch(endpoint).then(res => {
405             if (res.ok) setTimeout(updatePage, 100);
406             else { alert('Loi: ' + endpoint); updatePage(); }
407         });
408     }
409     updatePage();
410     setInterval(updatePage, 1000);
411 </script>
412 </body>
413 </html>
414 )rawliteral";
415
416 // === KHOI TAO ===
417 void setup() {
418     Serial.begin(115200);
419     pinMode(buzzerPin, OUTPUT);
420     pinMode(ledGreen, OUTPUT);
421     pinMode(ledYellow, OUTPUT);
422     pinMode(ledRed, OUTPUT);
423
424     digitalWrite(buzzerPin, HIGH); // Tat coi mac dinh
425
426     dht.begin();
427     lcd.init();
428     lcd.backlight();
429     lcd.clear();
430     lcd.setCursor(0, 0);
431     lcd.print("He thong: ON    ");
432     lcd.setCursor(0, 1);
433     lcd.print("                ");
434     delay(1000);
435
436     WiFi.softAP(ap_ssid, ap_password);
437     Serial.print("WiFi AP da tao: ");
438     Serial.println(ap_ssid);
439
440     mq4Value = analogRead(mq4Pin);
441     t = dht.readTemperature();
442     h = dht.readHumidity();
443     updateSystemStatus();
444     updateLCD();
445
446     server.on("/data", []() {
447         server.send(200, "application/json", getStatusJson());
448     });
449
450     server.on("/start", []() {
451         systemOn = true;
452         updateLCD();

```

```

453     server.send(200, "text/plain", "System started");
454 });
455
456 server.on("/stop", []() {
457     systemOn = false;
458     digitalWrite(buzzerPin, HIGH);
459     digitalWrite(ledGreen, LOW);
460     digitalWrite(ledYellow, LOW);
461     digitalWrite(ledRed, LOW);
462     lastBuzzerState = HIGH;
463     updateLCD();
464     server.send(200, "text/plain", "System stopped");
465 });
466
467 server.on("/", []() {
468     server.send_P(200, "text/html", webPageHtml);
469 });
470
471 server.begin();
472 }
473
474 // === VONG LAP CHINH ===
475 void loop() {
476     server.handleClient();
477     unsigned long currentMillis = millis();
478
479     // 1. Doc cam bien (Chu ky 1 giay)
480     if (currentMillis - lastSensorReadTime >= sensorReadInterval) {
481         lastSensorReadTime = currentMillis;
482         mq4Value = analogRead(mq4Pin);
483
484         float newT = dht.readTemperature();
485         float newH = dht.readHumidity();
486         if (!isnan(newT)) t = newT;
487         if (!isnan(newH)) h = newH;
488
489         updateSystemStatus();
490
491         if (systemOn) {
492             updateLCD();
493         }
494         Serial.printf("Metan: %d | Nhiet do: %.1fC | Do am: %.1f%%\n",
495             mq4Value, t, h);
496     }
497
498     // 2. Luan phien man hinh LCD moi 2 giay
499     if (currentMillis - lastLcdToggle >= lcdToggleInterval) {
500         lastLcdToggle = currentMillis;
501         showGasScreen = !showGasScreen;
502         if (systemOn) updateLCD();
503     }

```

```

503
504 // 3. Dieu khien thiet bi vat ly
505 if (systemOn) {
506     if (currentStatusClass == "safe") {
507         digitalWrite(ledGreen, HIGH);
508         digitalWrite(ledYellow, LOW);
509         digitalWrite(ledRed, LOW);
510         digitalWrite(buzzerPin, HIGH);
511         lastBuzzerState = HIGH;
512     }
513     else if (currentStatusClass == "warning") {
514         digitalWrite(ledGreen, LOW);
515         digitalWrite(ledYellow, HIGH);
516         digitalWrite(ledRed, LOW);
517
518         if (currentMillis - lastBuzzerBeep >= buzzerBeepInterval) {
519             lastBuzzerBeep = currentMillis;
520             lastBuzzerState = !lastBuzzerState;
521             digitalWrite(buzzerPin, lastBuzzerState);
522         }
523     }
524     else if (currentStatusClass == "danger") {
525         digitalWrite(ledGreen, LOW);
526         digitalWrite(ledYellow, LOW);
527         digitalWrite(ledRed, HIGH);
528         digitalWrite(buzzerPin, LOW);
529         lastBuzzerState = LOW;
530     }
531 } else {
532     digitalWrite(ledGreen, LOW);
533     digitalWrite(ledYellow, LOW);
534     digitalWrite(ledRed, LOW);
535     digitalWrite(buzzerPin, HIGH);
536 }
537 }

```